

Assembly Guide

Localization of a Mobile Robot

Author: Srikanth Nanda
Course of Studies: Business Intelligence and Data Analytics

First examiner: Prof. Dr. Elmar Wings

Second examiner: Prof. Dr. Jonas Schwarz

Contents

List of figures v

List of tables vii

Acronyms ix

1 Idea 1
- 1.1 Overview . . . 1
- 1.2 System Architecture . . . 1
- 1.3 Key Components . . . 1
- 1.4 System Architecture Overview . . . 1
- 1.5 Data Flow . . . 2
- 1.6 Component Placement . . . 3

2 Folder Structure 5
- 2.1 Repository Overview . . . 5
- 2.2 Top-Level Structure . . . 5
- 2.3 Assembly Directory . . . 5
  - 2.3.1 Purpose . . . 5
- 2.4 Code Directory . . . 5
  - 2.4.1 Purpose . . . 5
  - 2.4.2 Structure . . . 6
  - 2.4.3 JetBot . . . 6
  - 2.4.4 Solidworks . . . 6
- 2.5 JetsonNano Directory . . . 6
  - 2.5.1 Purpose . . . 6
  - 2.5.2 Structure . . . 6
- 2.6 Manual Directory . . . 7
  - 2.6.1 Purpose . . . 7
- 2.7 MLbib Directory . . . 7
  - 2.7.1 Purpose . . . 7
- 2.8 Thesis Directory . . . 8
  - 2.8.1 Purpose . . . 8
- 2.9 Presentations Directory . . . 8
  - 2.9.1 Purpose . . . 8
- 2.10 Poster Directory . . . 8
  - 2.10.1 Purpose . . . 8

2.11 Project Management Directory 9
2.11.1 Purpose 9
2.12 Directory Summary 9

3 Files **11**
3.1 Overview 11
3.2 Documentation Files 11
3.3 Software Files 11
3.3.1 JetBot Software 11
3.3.2 Configuration and Launch Files 12
3.4 Implementation Files 12
3.5 Hardware and CAD Files 12
3.6 Bibliography Files 13
3.7 Data and Image Files 13
3.8 Manual Files 13
3.9 Presentation Files 13
3.10 Build Files 14
3.11 File Management 14

4 Manufacturing Hints **15**
4.1 Introduction 15
4.2 Platform Overview 15
4.3 Assembly and Manufacturing Considerations 16
4.3.1 Motor Orientation and Polarity 16
4.3.2 Standoff Set Confusion 16
4.3.3 Camera Ribbon Cable Fragility 16
4.3.4 Battery Pack Mounting 16
4.3.5 Wiring and Tolerance Management 17
4.4 Design Decision: Auxiliary Bumper Sensor Integration 17
4.4.1 Rationale 17
4.4.2 Integration Considerations 18
4.4.3 Methodological Boundary 18
4.5 Mechanical Bumper Design and Integration 18
4.6 Design Rationale 18
4.7 Materials and Construction 19
4.8 Mounting on the Chassis 20
4.9 Electrical Integration 20
4.10 Software Integration with the Collision-Avoidance Pipeline 20

5 Assembly **23**
5.1 Assembly 23
5.2 Assembly Best Practices 26
5.3 Component Verification 26

5.4 Component Functions in Localization Context . . . 27
5.4.1 Camera Module . . . 27
5.4.2 Motor and Encoder System . . . 27
5.4.3 Jetson Nano Compute Platform . . . 27
5.5 Sensor Synchronization . . . 27
5.5.1 Next Steps . . . 27

6 Maintenance **29**
6.1 Routine Hardware Maintenance . . . 29
6.1.1 Camera Lens Care . . . 29
6.1.2 Motor and Encoder Service . . . 29
6.1.3 Structural Inspection . . . 29
6.2 Battery Management . . . 29

7 Safety Guidelines **31**
7.1 Assembly Safety . . . 31
7.2 Operational Safety . . . 31
7.3 Electrical Safety . . . 31
7.4 Battery Handling . . . 31

8 Troubleshooting **33**
8.1 Camera Issues . . . 33
8.2 Motor Problems . . . 33
8.3 Electrical Issues . . . 33

9 Add Ons **35**
9.1 Optional Sensor Expansions . . . 35
9.1.1 9-DOF IMU Module (LSM9DS1) . . . 35
9.1.2 Stereo Camera Setup (Dual Camera) . . . 35
9.2 Computing Upgrades . . . 35
9.3 Power Enhancement . . . 35
9.3.1 JetPack 4.5 Additions over JetPack 4.2 . . . 36

Bibliography **39**

List of Figures

4.1 Bumper for Mobile Robot 19

5.1 Main mechanical and electronic parts of the SparkFun JetBot AI Kit before assembly.[Spa20a] 23
5.2 Top-plate accessory installation showing Leopard Imaging IMX 219 Camera and cable routing.[Spa20a] 24
5.3 Front camera bracket and CSI ribbon installation on the JetBot chassis.[Spa20a] 24
5.4 Mounting Battery [Spa20a] 25
5.5 Fully assembled SparkFun JetBot AI Kit used as the mobile robot platform [Spa20a]. 26

List of Tables

2.1 Repository directory overview. 9

9.1 Core library versions: JetPack 4.2 vs. JetPack 4.5 36

Acronyms

DC Direct Current

GPU Graphics Processing Unit

IMU Inertial Measurement Unit

OLED Organic Light-Emitting Diode

RGB Rot-Grün-Blau

USB Universal Serial Bus

1 Idea

1.1 Overview

This documentation provides comprehensive instructions for assembling a Jetson Nano-based mobile robot platform and the files and folder structure implementation designed for localization research and testing. The robot integrates visual odometry, sensor fusion, and simultaneous localization and mapping (SLAM) capabilities.

1.2 System Architecture

The mobile robot consists of:

- NVIDIA Jetson Nano development platform (compute core)
- Dual-motor transmission system with encoders
- RGB camera for visual odometry and environment mapping
- Motor drivers and power management circuits
- Support for optional IMU and additional sensors

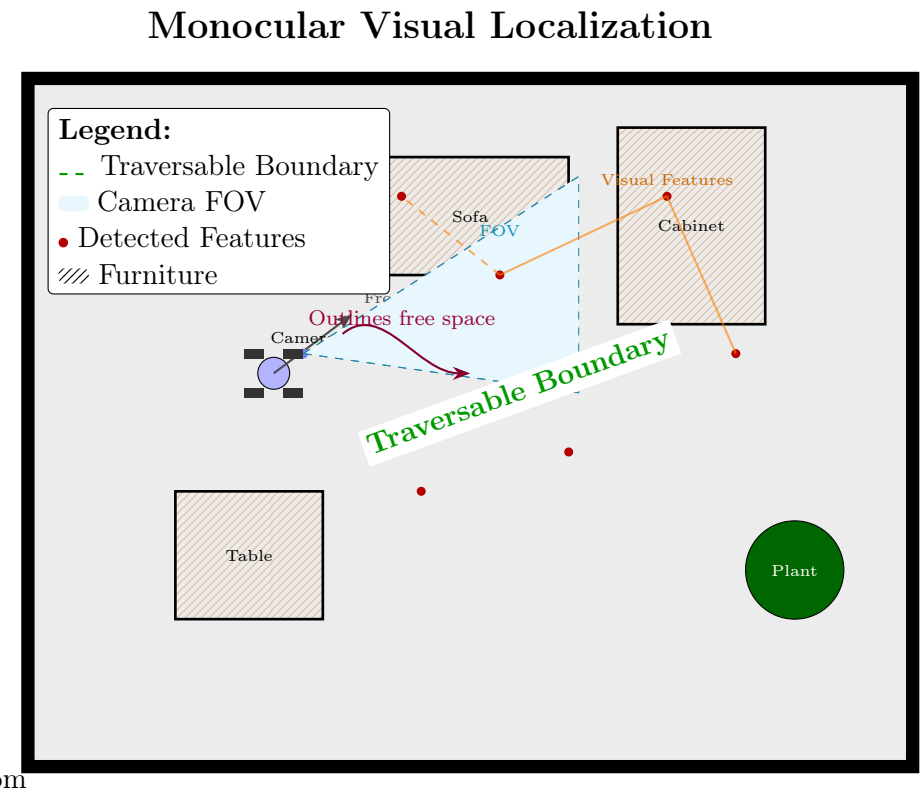
1.3 Key Components

Each component plays a critical role in the localization pipeline. The camera provides environmental visual features, motors enable precise movement, and encoders provide odometry feedback for trajectory estimation.

1.4 System Architecture Overview

The mobile robot platform integrates:

1. **Perception**: RGB camera for visual feature extraction
2. **Preconception**: Motor encoders for odometry feedback
3. **Computation**: Jetson Nano GPU processing visual and encoder data
4. **Actuation**: Dual motor + encoder system for differential drive

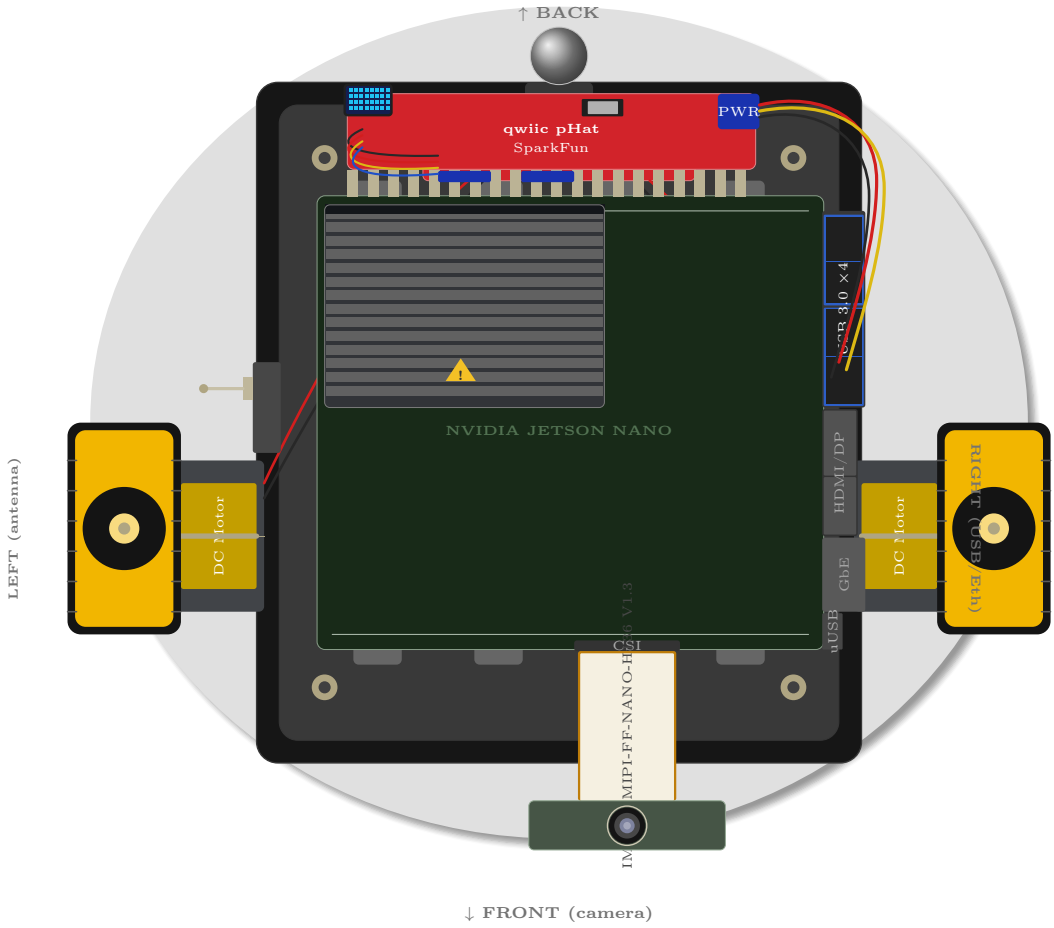


Room

1.5 Data Flow

- Camera captures 30 frames per second at 640x480 resolution
- Encoders provide tick-based feedback at motor control frequency
- All streams are timestamped and synchronized
- Feature-to-feature correspondences drive motion estimation

1.6 Component Placement



SparkFun JetBot AI Kit — Top View

2 Folder Structure

2.1 Repository Overview

The repository is divided into directories for hardware documentation, software, Jetson Nano development, manuals, references, thesis material, presentations, and project management.

2.2 Top-Level Structure

```
250901-Jetson-Nano-Ground/
|-- README.md
|-- author.xlsx
|-- Assembly/
|-- Code/
|-- JetsonNano/
|-- Manual/
|-- MLbib/
|-- Thesis/
|-- Presentations/
|-- Poster/
‘-- ProjectManagement/
```

2.3 Assembly Directory

2.3.1 Purpose

The `Assembly/` directory contains hardware documentation, assembly instructions, and related LaTeX files.

2.4 Code Directory

2.4.1 Purpose

The `Code/` directory contains the main software implementations, microcontroller programs, and CAD-related files.

2.4.2 Structure

```
Code/
|-- JetBot/              # Robot control software
|-- DomainSystem/        # Domain-specific modules
|-- MicroPython/         # MicroPython programs
‘-- Solidworks/          # CAD models and G-code files
```

2.4.3 JetBot

```
Code/JetBot/
|-- CMakeLists.txt
|-- package.xml
|-- README.md
|-- requirements.txt
|-- run_jetbot_all.sh
|-- config/              # YAML configuration files
|-- launch/              # ROS2 launch files
|-- models/              # Trained ML models
|-- scripts/             # Python and shell scripts
‘-- src/                 # Optional C++ source files
```

2.4.4 Solidworks

```
Code/Solidworks/
|-- RobotBumper.SLDPRT
|-- RobotBumper.3mf
|-- RobotBumper2.SLDPRT
|-- RobotBumper2.3mf
|-- RobotBumper_0.2mm_PLA_MK3_45m.gcode
‘-- RobotBumper2_0.25n_0.1mm_ABS_MK3S_3h13m.gcode
```

2.5 JetsonNano Directory

2.5.1 Purpose

The `JetsonNano/` directory contains Jetson Nano-specific code, system configuration, datasets, images, and experimental results.

2.5.2 Structure

```
JetsonNano/
|-- Code/
|   |-- JetsonNano/          # Localization and avoidance code
```

```
|   |-- Arduino/              # Arduino integration
|   |-- TensorFlowLite/       # TFLite inference scripts
|   `-- Nano33BLESense/       # Sensor support
|-- System/                   # Setup and system configuration
|-- Contents/                 # Datasets, calibration data, logs
|-- Images/                   # Captured images and results
|-- General/                  # Documentation
`-- tikz/                     # TikZ diagrams
```

2.6 Manual Directory

2.6.1 Purpose

The `Manual/` directory contains instructions for users and developers.

```
Manual/
|-- DeveloperManual/
|   |-- Environment_Setup.md
|   |-- Building_and_Testing.md
|   |-- Debugging_Guide.md
|   |-- API_Reference.md
|   `-- Contributing.md
`-- UserManual/
|-- Hardware_Assembly.md
|-- Quick_Start.md
|-- Operation_Guide.md
|-- Calibration.md
|-- Safety_Guidelines.md
`-- Troubleshooting.md
```

2.7 MLbib Directory

2.7.1 Purpose

The `MLbib/` directory contains BibTEX references for the thesis, documentation, and presentations.

```
MLbib/
|-- Jetson.bib
|-- Literature.bib
|-- MyLiterature.bib
|-- Localization/
|-- ML/
|-- MobileRobot/
```

```
|-- Manuals/
‘-- SLAM/
```

2.8 Thesis Directory

2.8.1 Purpose

The `Thesis/` directory contains the main thesis document, chapters, images, references, and build files.

```
Thesis/
|-- LocalizationofaMobileRobot.tex
|-- LocalizationofaMobileRobot.pdf
|-- Contents/              # Thesis chapters
|-- General/               # Preamble, packages, commands
|-- Images/                # Figures, diagrams, plots, results
|-- tikz/                  # TikZ figures
‘-- [build artifacts]      # .aux, .toc, .lof, etc.
```

2.9 Presentations Directory

2.9.1 Purpose

The `Presentations/` directory contains slides and presentation material.

```
Presentations/
|-- presentationinfo.txt
|-- Localization/
|-- SLAM/
|-- ROS2/
|-- NVIDIAJetsonNano/
|-- MobileRobot/
|-- LiteratureReview/
|-- Manual/
‘-- ThesisColloquiumPresentation/
```

2.10 Poster Directory

2.10.1 Purpose

The `Poster/` directory contains the academic poster source files and generated PDF.

```
Poster/
|-- tikzposter.tex
|-- tikzposter.pdf
|-- images/
‘-- [build artifacts]
```

2.11 Project Management Directory

2.11.1 Purpose

The `ProjectManagement/` directory contains planning and organisational documents, such as:

- Project timelines and milestones.
- Meeting notes.
- Task planning and progress reports.
- Resource and schedule documents.

2.12 Directory Summary

Directory	**Purpose**
Assembly	Hardware documentation and assembly material
Code	Robot software, microcontroller code, and CAD files
JetsonNano	Jetson Nano code, configuration, datasets, and results
Manual	User and developer documentation
MLbib	BibTEX literature references
Thesis	Thesis source files, figures, and compiled documents
Presentations	Presentation slides and supporting material
Poster	Academic poster source and PDF
ProjectManagement	Project planning and progress documents

Table 2.1: Repository directory overview.

“‘

“‘latex

3 Files

3.1 Overview

This chapter describes the main files in the repository.

3.2 Documentation Files

- **README.md**: Main project overview and repository guide.
- **author.xlsx**: Author and contributor information.
- **Thesis/LocalizationofaMobileRobot.tex**: Main thesis source file.
- **Thesis/LocalizationofaMobileRobot.pdf**: Compiled thesis document.
- **Assembly/MobileRobotAssembly.tex**: Hardware assembly documentation.
- **Assembly/MobileRobotAssembly.pdf**: Compiled assembly manual.
- **Poster/tikzposter.tex**: Academic poster source file.
- **Poster/tikzposter.pdf**: Compiled academic poster.

3.3 Software Files

3.3.1 JetBot Software

The main robot-control software is located in `Code/JetBot/`.

- **CMakeLists.txt**: CMake build configuration.
- **package.xml**: ROS2 package information and dependencies.
- **requirements.txt**: Required Python packages.
- **run_jetbot_all.sh**: Script to start the robot system.
- **README.md**: JetBot setup and usage instructions.

3.3.2 Configuration and Launch Files

- **Code/JetBot/config/**: Robot parameters, calibration values, and sensor settings.
- **Code/JetBot/launch/**: ROS2 launch files for starting robot nodes.
- **Code/JetBot/scripts/**: Python and shell scripts for experiments and utilities.
- **Code/JetBot/models/**: Trained machine-learning models.

3.4 Implementation Files

- **Code/DomainSystem/Test.py**: Test and validation script.
- **Code/MicroPython/**: MicroPython programs for microcontrollers.
- **JetsonNano/Code/JetsonNano/**: Localization and collision-avoidance implementations.
- **JetsonNano/Code/Arduino/**: Arduino integration code.
- **JetsonNano/Code/TensorFlowLite/**: TensorFlow Lite inference scripts.
- **JetsonNano/Code/Nano33BLESense/**: Arduino Nano 33 BLE Sense support code.

3.5 Hardware and CAD Files

The hardware design files are stored in `Code/Solidworks/`.

- **RobotBumper.SLDPRT**: Main bumper CAD model.
- **RobotBumper2.SLDPRT**: Alternative bumper CAD model.
- **RobotBumper.3mf**: 3D-print model export.
- **RobotBumper2.3mf**: Alternative 3D-print model export.
- **RobotBumper_0.2mm_PLA_MK3_45m.gcode**: G-code for PLA printing.
- **RobotBumper2_0.25n_0.1mm_ABS_MK3S_3h13m.gcode**: G-code for ABS printing.

3.6 Bibliography Files

The bibliography files are stored in `MLbib/`.

- **Jetson.bib**: NVIDIA Jetson references.
- **Literature.bib**: General academic references.
- **MyLiterature.bib**: Custom bibliography entries.
- **Literatur.bib**: German-language references.
- **MLbib/Localization/**: Localization references.
- **MLbib/ML/**: Machine-learning references.
- **MLbib/MobileRobot/**: Mobile robotics references.
- **MLbib/SLAM/**: SLAM-related references.
- **MLbib/Manuals/**: Hardware and software manuals.

3.7 Data and Image Files

- **JetsonNano/Contents/**: Datasets, calibration files, logs, and experiment results.
- **JetsonNano/Images/**: Images and visual results from robot experiments.
- **Thesis/Images/**: Figures, diagrams, and plots used in the thesis.
- **Assembly/Images/**: Hardware photographs and assembly diagrams.

3.8 Manual Files

- **Manual/DeveloperManual/**: Setup, build, debugging, and contribution guides.
- **Manual/UserManual/**: Assembly, operation, calibration, safety, and troubleshooting guides.

3.9 Presentation Files

- **Presentations/presentationinfo.txt**: Presentation index and metadata.
- **Presentations/**: Topic-specific presentation directories.
- **Thesis/tikz/**: TikZ graphics for the thesis.
- **JetsonNano/tikz/**: TikZ diagrams for Jetson Nano documentation.

3.10 Build Files

LaTeX creates build files during compilation. These files can usually be deleted and regenerated.

- ***.aux**: Auxiliary reference information.
- ***.toc**: Table of contents data.
- ***.lof**: List of figures data.
- ***.lot**: List of tables data.
- ***.bbl**: Generated bibliography.
- ***.log**: Compilation log file.
- ***.pdf**: Final compiled document.

3.11 File Management

- Store source code in `Code/` and `JetsonNano/Code/`.
- Store documentation in `Manual/`, `Assembly/`, and `Thesis/`.
- Store references in `MLbib/`.
- Store experimental data and images in `JetsonNano/Contents/` and `JetsonNano/Images/`.
- Commit source files to version control and exclude temporary build files where possible.

‘‘‘

4 Manufacturing Hints

4.1 Introduction

The experimental validation of the monocular visual SLAM (vSLAM) and collision-avoidance pipeline proposed in this thesis relies on a physical mobile robot testbed rather than a purely simulated environment. This chapter documents the hardware platform used throughout the experiments, the manufacturing and assembly considerations that influenced build quality and repeatability, and the design decision to augment the platform with a supplementary mechanical bumper sensor. The purpose of this chapter is twofold: first, to provide a reproducible account of the physical build process so that the platform can be replicated or extended by future work; second, to justify architectural decisions, particularly the bumper integration that preserves the methodological integrity of the thesis's core claim, namely that collision avoidance can be achieved using a *monocular-only* perception stack.

4.2 Platform Overview

The testbed is a SparkFun JetBot v2.1-based differential-drive mobile robot built around the NVIDIA Jetson Nano Developer Kit A02 [NVI19; NVI20]. The platform consists of the following principal subsystems:

- **Compute:** NVIDIA Jetson Nano Developer Kit A02, providing GPU-accelerated inference for the CNN-based collision-avoidance classifier and CPU cycles for the ORB-SLAM2/3-based localization pipeline [NVI19].
- **Perception:** A single front-mounted CSI camera. The Jetson Nano A02 exposes only one CSI camera interface, which is the primary hardware constraint motivating the monocular (rather than stereo or RGB-D) formulation of the SLAM problem in this thesis.
- **Actuation:** A two-motor differential-drive chassis controlled through a Qwiic-interfaced motor driver, communicating with the Jetson Nano over I2C [Spa19a].
- **Power:** A single-cell battery pack supplying both the Jetson Nano carrier and the motor driver, imposing a realistic constraint on continuous operating time and peak current draw during simultaneous inference and motor actuation.

This configuration positions the platform as a resource-constrained embedded system representative of low-cost indoor service robots [Spa20b], rather than a laboratory rig with unconstrained compute or power budgets.

4.3 Assembly and Manufacturing Considerations

The chassis and mechanical hardware follow SparkFun's JetBot AI Kit assembly documentation, the most recent published revision of which is the v2.0 guide; this remains functionally applicable to the v2.1 platform used here, as the chassis, motor, and camera mounting geometry are unchanged between revisions [Spa20a]. Several manufacturing and assembly considerations proved consequential for build integrity and are recorded here for reproducibility.

4.3.1 Motor Orientation and Polarity

Both DC gearmotors must be installed with their identifying sticker facing outward from the chassis. This orientation is not cosmetic: it determines whether the motor leads map correctly onto the Qwiic Motor Driver's A1/A2 (left channel) and B1/B2 (right channel) terminals [Spa19a]. An inverted motor produces a robot that yaws or spins rather than tracking a straight commanded trajectory, which would otherwise be misinterpreted as an odometry or control-loop defect during early testing [Spa20a].

4.3.2 Standoff Set Confusion

The chassis kit ships two dimensionally similar sets of brass standoffs — one for the main chassis frame and a shorter set for the caster ball assembly. Because both sets are packaged together, they are easily interchanged during assembly. Using the wrong set alters the robot's ground clearance and pitch, which in turn shifts the camera's fixed viewing angle relative to the floor plane — a parameter that matters directly for this thesis, since the collision-avoidance CNN and the ORB feature extractor are both sensitive to the camera's height and tilt relative to the scene [Spa20a].

4.3.3 Camera Ribbon Cable Fragility

The CSI ribbon cable connector on the Jetson Nano is a plastic latch mechanism that degrades with repeated insertion cycles and is not field-replaceable. Given that this thesis's entire perception stack — both localization and obstacle detection — depends on a single camera feed, the ribbon cable represents a single point of failure for the whole system. The cable must be inserted with the correct half-twist so that the contact traces align, and re-seating should be minimized once a working configuration is confirmed [Spa20a].

4.3.4 Battery Pack Mounting

In the v2.0 revision, SparkFun relocated the battery mount from the underside of the chassis to the top plate, because the adhesive mounting strip was insufficiently strong to retain the pack underneath during motion [Spa20a]. The battery and its leads must be routed to avoid fouling the camera mount or overhanging the rear axle,

both of which would alter the robot's center of mass and, consequently, its dynamics during differential-drive maneuvers.

4.3.5 Wiring and Tolerance Management

Motor leads should be routed through the baseplate cutouts before the chassis is closed, and slack should be secured around a standoff rather than left loose near the wheel wells, where it can be caught during rotation [Spa20a]. The Jetson Nano's mounting holes are oblong to absorb manufacturing tolerance; standoffs on one side of the board should remain loosely fastened until the board is fully seated, to avoid inducing mechanical stress on the carrier board [NVI20].

4.4 Design Decision: Auxiliary Bumper Sensor Integration

4.4.1 Rationale

Although the thesis's central contribution is a monocular vision-based collision-avoidance pipeline, a supplementary mechanical bumper switch was integrated into the platform as an auxiliary safety and evaluation component. This decision is justified on three grounds.

First, **fail-safe redundancy**. A CNN-based obstacle classifier operating on a single monocular feed is subject to known failure modes, low-illumination frames, textureless or specular surfaces, and objects entering the scene from outside the camera's instantaneous field of view immediately before contact. Vision-based perception in cluttered and dynamic indoor environments remains an open robustness challenge [CCC+16; MBR17], motivating a hardware-level stop signal that is independent of the inference pipeline and therefore immune to these failure modes.

Second, **resource independence**. The bumper is read as a simple digital GPIO input and does not contend with the shared CSI camera resource used by both the vSLAM and collision-avoidance modules. Given the limited compute and memory budget of lightweight CNN models deployed on resource-constrained edge platforms [Zuf+24], its inclusion adds redundancy without imposing additional inference or camera-bandwidth load on the perception pipeline.

Third, **quantitative evaluation utility**. A bumper contact event constitutes an unambiguous ground-truth signal that a physical collision has occurred. Logging bumper triggers alongside the CNN's "blocked" predictions enables direct computation of the false-negative rate of the vision-based collision-avoidance classifier is concrete, reproducible metric reported in the results chapter of this thesis.

4.4.2 Integration Considerations

The bumper switch is mounted around the front perimeter of the chassis, positioned to avoid occluding the camera's field of view. It is wired to a Jetson Nano GPIO pin with a pull-up resistor and software debounce logic, triggering an immediate motor-stop interrupt that operates independently of the main perception and control loop [NVI20]. This ensures that even a stalled or delayed inference cycle does not prevent an emergency stop.

4.4.3 Methodological Boundary

The bumper sensor is explicitly treated as an auxiliary safety and evaluation instrument, not as a component of the core perception architecture under study. This distinction is maintained throughout the thesis to avoid diluting the central research claim — that indoor obstacle avoidance can be achieved using monocular vision alone [MBR17], as constrained by the single CSI camera interface of the Jetson Nano Developer Kit A02 [NVI19]. All reported collision-avoidance performance metrics are computed from the CNN's vision-based predictions; the bumper is used only for ground-truth verification and as a safety backstop during physical trials, and its activation is reported separately from the vision-only evaluation results.

4.5 Mechanical Bumper Design and Integration

While the monocular visual SLAM and CNN-based obstacle-avoidance pipeline described in the preceding chapters provide the robot's primary means of perceiving and reacting to obstacles, both approaches are fundamentally predictive and vision-dependent: they can fail under motion blur, poor lighting, low-texture surfaces, or objects that enter the camera's blind spot close to the chassis. To guard against these failure modes, a purely mechanical, contact-based bumper was designed and fabricated from scratch and integrated directly with the Jetson Nano Developer Kit A02 as a hardware-level, last-resort collision-detection mechanism. This chapter documents the design rationale, physical construction, mounting, electrical integration, and software-level role of this bumper within the overall collision-avoidance pipeline.

4.6 Design Rationale

The SparkFun JetBot v2.1 platform, as configured for this thesis, does not include an inertial measurement unit (IMU), wheel encoders, or any dedicated proximity sensor (e.g., ultrasonic or infrared rangefinders). Consequently, the only sources of environmental information available to the robot are the single front-facing monocular camera feed, which is shared between the vSLAM localization module and the CNN-based obstacle-detection module. Since both of these modules operate on the same visual stream and both are susceptible to correlated failure modes (e.g., sudden

illumination changes, featureless walls, or fast-approaching obstacles at close range where perspective cues collapse), a physically independent, sensor-diverse fallback was considered necessary to prevent hard collisions that could damage the chassis, the camera mount, or the motor drivetrain.

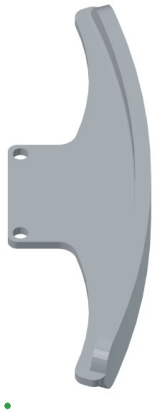


Figure 4.1: Bumper for Mobile Robot

A mechanical, switch-based bumper was chosen over an additional active sensor (such as an ultrasonic or infrared ranger) for three reasons. First, it requires no additional calibration or software filtering, since it produces a binary contact signal that is trivial to interpret. Second, it consumes negligible power and computational resources, which is important given the constrained power budget of the single-cell battery pack. Third, it is entirely independent of lighting conditions and surface texture, making it complementary rather than redundant with respect to the vision-based pipeline.

4.7 Materials and Construction

The bumper structure was assembled entirely from off-the-shelf and 3D-printed components, with no pre-made bumper kit used. The main structural element is a semi-circular ring fabricated from polylactic acid filament using fused deposition modeling (FDM) 3D printing, chosen for its rigidity, ease of printing, and low weight relative to the JetBot chassis. The ring was designed to wrap around the front half of the chassis perimeter, covering the region most likely to make first contact with obstacles given the robot's differential-drive kinematics and forward-biased camera field of view.

Two miniature tactile lever-arm microswitches were embedded at the leading edge of the bumper ring, spaced symmetrically about the chassis centerline to allow the system to distinguish between a left-side, right-side, or head-on collision based on

which switch (or both) is triggered. A thin compliant foam strip was affixed along the outer edge of the printed ring to absorb minor impacts, provide a small amount of pre-travel before the microswitches engage, and reduce the shock transmitted to the printed structure itself.

4.8 Mounting on the Chassis

The bumper assembly is mounted at the front of the JetBot chassis, ahead of and below the camera mount, so that it does not obstruct the monocular camera's field of view while still providing forward impact protection. The printed ring is secured to the existing chassis standoff holes using M3 nylon screws and standoffs, chosen to keep the added mass low and to avoid interfering with the Qwiic motor controller and wiring harness located on the chassis underside. This mounting position was selected after considering the robot's turning behavior: because the differential-drive base can pivot in place, the bumper's semi-circular coverage ensures that both frontal and near-frontal oblique collisions are detected regardless of the robot's instantaneous heading during an avoidance maneuver.

4.9 Electrical Integration

Each microswitch is wired as a simple normally-open contact switch, pulled high through an internal pull-up configuration and connected directly to a dedicated GPIO pin on the Jetson Nano Developer Kit A02's 40-pin expansion header. This keeps the bumper entirely independent of the Qwiic I2C bus used for the motor controller, so that a bumper-triggered event can be read and acted upon even if the I2C bus is congested or temporarily unresponsive. When a switch closes on contact, the corresponding GPIO line is pulled low, generating a signal that is polled by a lightweight background thread on the Jetson Nano.

No additional signal conditioning circuitry was required beyond basic debouncing, which is handled in software (Section 4.10) rather than with discrete RC filtering hardware, in order to keep the electrical design as simple and failure-resistant as possible.

4.10 Software Integration with the Collision-Avoidance Pipeline

At the software level, the bumper is treated as a hard interrupt source with the highest priority in the overall collision-avoidance stack. A dedicated GPIO polling routine, running independently of the vSLAM and CNN inference threads, continuously monitors both microswitch lines at a fixed sampling rate and applies a short software debounce window to filter out spurious transitions caused by vibration during normal driving.

Whenever a bumper contact event is confirmed, the routine immediately issues a motor-stop command directly to the Qwiic motor controller, pre-empting any velocity command currently being issued by the CNN-based obstacle-avoidance policy or the vSLAM-driven navigation logic. Following a stop, the robot executes a short pre-programmed reverse-and-turn maneuver away from the side on which contact was detected (left switch, right switch, or both, in the head-on case) before control is returned to the primary vision-based pipeline. This ensures that the mechanical bumper functions strictly as a safety net beneath the monocular vSLAM and CNN-based obstacle-avoidance system, intervening only when the vision-based pipeline has already failed to prevent a physical collision.

5 Assembly

5.1 Assembly

The mobile robot is assembled following the SparkFun JetBot AI Kit v2.0 hardware guide, which describes how to mount the NVIDIA Jetson Nano A02, camera, battery, and accessories on the two-plate chassis [Spa20a]. The lower plate holds the DC motors, wheels, and basic chassis structure, while the upper plate carries the NVIDIA Jetson Nano A02 Developer Kit and Qwiic expansion hardware [Spa20a]. Assembly uses standard M3 hardware and a small Phillips screwdriver, and no soldering is required because all electrical interconnects are provided through Qwiic cables and ribbon connectors [Spa20a].

The camera assembly step mounts the Leopard Imaging IMX219 Camera module onto the front camera bracket using the supplied screws and standoffs [Spa20a]. The bracket is then fixed to the front edge of the top plate so that the camera optical axis points forward and slightly downward toward the floor [Spa20a].



Figure 5.1: Main mechanical and electronic parts of the SparkFun JetBot AI Kit before assembly.[Spa20a]

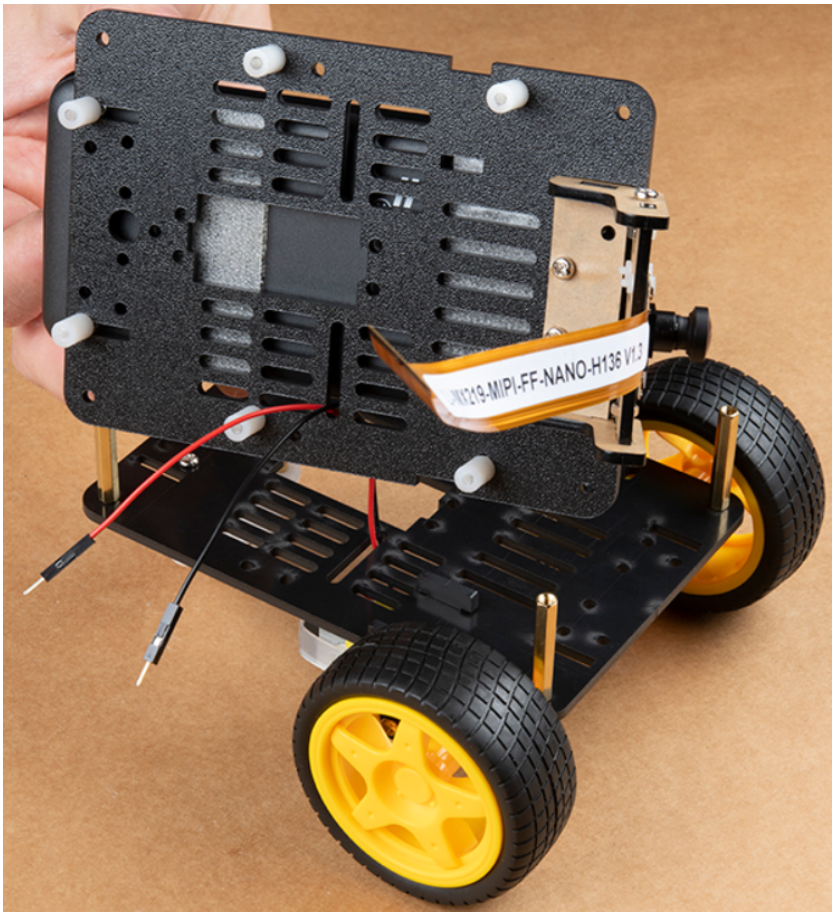


Figure 5.2: Top-plate accessory installation showing Leopard Imaging IMX 219 Camera and cable routing.[Spa20a]



Figure 5.3: Front camera bracket and CSI ribbon installation on the JetBot chassis.[Spa20a]

The CSI ribbon cable is routed from the camera to the NVIDIA Jetson Nano A02 camera connector, taking care to avoid sharp bends and to maintain the correct connector orientation [NVI19]. Cable slack is managed so that the ribbon does not obstruct the field of view or interfere with wheel motion [Spa20a].



Figure 5.4: Mounting Battery [Spa20a]

The Eunicell USB power bank is mounted on the top plate in the updated assembly configuration rather than underneath the chassis [Spa20a]. This placement improves access to the power button and charge port and avoids reducing ground clearance.[Spa20a] Velcro straps or elastic bands secure the battery pack to the top plate so that it cannot slide during acceleration or collision tests. [Spa20a]. The USB cable from the battery pack is plugged into the Jetson Nano power input as specified in the assembly guide to provide sufficient current during peak load [Spa20a].

Accessory installation continues with mounting the SparkFun Qwiic Motor Driver and SparkFun Micro OLED Breakout on the top plate around the NVIDIA Jetson Nano A02 [Spa20a]. The Qwiic pHAT is connected to the 40-pin header on the NVIDIA Jetson Nano A02 and secured using the included standoffs so that Qwiic connectors remain accessible [Spa19b]. Short Qwiic cables connect the pHAT to the motor driver and the Micro OLED, forming the local I^2C interconnect on the upper deck [Spa20a]. The motor driver output terminals are wired to the left and right DC motors, and polarity is selected so that positive commands correspond to forward motion on both sides [Spa19a].

Once all components are mounted, the top plate is attached to the lower chassis using the supplied standoffs, producing a rigid two-level robot structure [Spa20a]. Final checks include verifying that all screws are tightened, wheels rotate freely, connectors are seated properly, and the camera view remains unobstructed [SNS11].

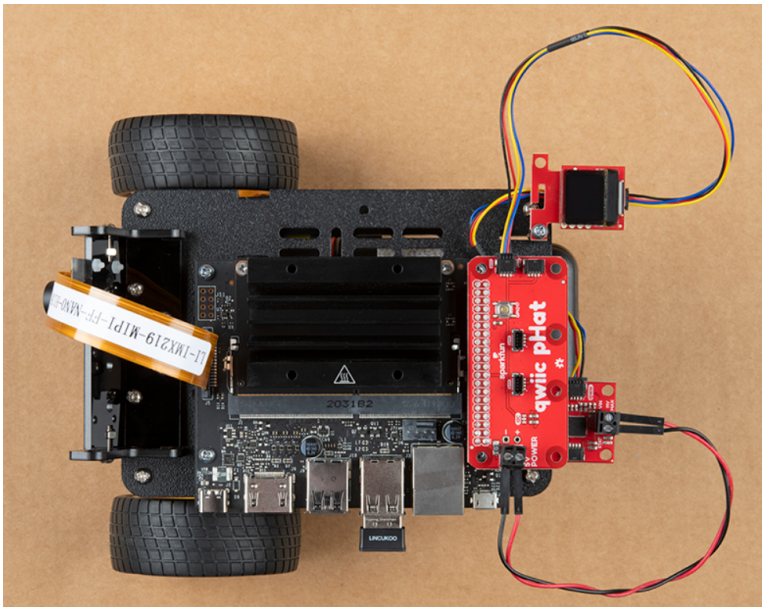


Figure 5.5: Fully assembled SparkFun JetBot AI Kit used as the mobile robot platform [Spa20a].

Only after the mechanical assembly passes these checks is the system powered on for software setup and calibration [Spa20a].

5.2 Assembly Best Practices

- Ensure camera lens is clean and properly focused before assembly
- Mount camera on a stable platform to minimize vibration during motion
- Verify motor encoder alignments for accurate odometry
- Use vibration-damping materials between compute platform and chassis
- Keep cable routing organized to avoid mechanical interference
- Test all connections before final enclosure

5.3 Component Verification

- Motor responses should be immediate and smooth
- Camera stream should display without artifacts or lag
- Encoder pulses must be consistent during calibration runs
- Power delivery should be stable under full load

5.4 Component Functions in Localization Context

5.4.1 Camera Module

- Captures 30 fps video stream for feature extraction
- Provides environmental context for visual odometry
- Enables loop closure detection through image similarity

5.4.2 Motor and Encoder System

- Propels robot along desired trajectory
- Encoders count wheel rotations for incremental odometry
- Enables dead reckoning between visual updates

5.4.3 Jetson Nano Compute Platform

- Processes images at real-time frame rate
- Solves pose graph optimization problems
- Maintains feature descriptor database for loop closure

5.5 Sensor Synchronization

All sensors must be synchronized to timestamp events correctly. Motor encoders provide synchronous pulses; camera frames are timestamped on capture.

5.5.1 Next Steps

Follow Domain Hardware and Domain Software in Thesis Report for more guided Software installation, testing and calibration of the Mobile robot.

6 Maintenance

6.1 Routine Hardware Maintenance

6.1.1 Camera Lens Care

- Clean lens weekly with lens cloth and appropriate cleaning solution
- Check focus clarity by capturing test images
- Replace lens if scratches affect image quality

6.1.2 Motor and Encoder Service

- Inspect wheelbase for debris or buildup (monthly)
- Test motor smoothness and listen for grinding sounds
- Check encoder pulse consistency with test run
- Replace motor brushes if performance degrades

6.1.3 Structural Inspection

- Check all fasteners for looseness after high-acceleration runs
- Inspect caster ball for wear
- Verify camera mount rigidity

6.2 Battery Management

- Recharge after each use session
- Check for swelling or damage
- Replace if capacity drops below 80%

7 Safety Guidelines

7.1 Assembly Safety

- Disconnect battery before making electrical connections
- Wear safety goggles when working with power tools
- Do not test motors with fingers near moving parts
- Use anti-static wrist strap when handling Jetson Nano

7.2 Operational Safety

- Operate robot only in clear, obstacle-free areas during testing
- Never reach into or block the robot while motors are powered
- Keep long hair and loose clothing away from spinning wheels
- Maintain full visibility of robot trajectory during autonomous operation

7.3 Electrical Safety

- Use approved power supply only; do not attempt to modify voltage
- Check for exposed wires; insulate with tape if necessary
- Do not operate in wet environments
- Allow system to cool if running continuously longer than 1 hour

7.4 Battery Handling

- Do not short-circuit battery terminals
- Do not dispose of battery in regular trash; use proper recycling
- Stop use immediately if battery shows signs of damage, swelling, or overheating

8 Troubleshooting

8.1 Camera Issues

Problem	Cause	Solution
No video stream	Loose CSI ribbon cable	Reseat cable in Jetson connector
Blurry images	Lens out of focus or dirty	Clean lens; adjust focus ring
Video freezes	Driver issue or overheating	Reboot system; check temperature
Discolored frame	Cable bent or damaged	Replace CSI cable carefully

8.2 Motor Problems

- **No motor response**: Check battery voltage; verify GPIO connections
- **Jerky motion**: Loose wheel; recheck fasteners
- **Encoder not counting**: Misaligned encoder; clean optical surfaces

8.3 Electrical Issues

- **Brown-out**: Battery voltage too low; recharge immediately
- **Jetson won't boot**: Check power supply continuity
- **Motors stall**: Reduce load or check for mechanical binding

9 Add Ons

9.1 Optional Sensor Expansions

9.1.1 9-DOF IMU Module (LSM9DS1)

- Provides accelerometer, gyroscope, magnetometer data
- Enables heading estimation independent of visual features
- Improves odometry during rapid rotations
- Connection: I2C pins on Jetson Nano GPIO (pins 3, 5)

9.1.2 Stereo Camera Setup (Dual Camera)

- Enables depth estimation for 3D mapping
- Provides baseline for triangulation-based localization
- Higher computational cost but better accuracy in textured environments
- Requires separate CSI camera interface or USB hub

9.2 Computing Upgrades

- **Jetson Xavier NX**: 8x more performance, enables heavier algorithms
- **Jetson Orin Nano**: Latest architecture, 4x faster than Nano
- Note: Code compatibility maintained across Jetson family

9.3 Power Enhancement

- Upgrade to higher-capacity battery (20000 mAh) for extended runtime
- Add on-robot charging dock for autonomous recharging
- Enable continuous operation for multi-hour deployments

9.3.1 JetPack 4.5 Additions over JetPack 4.2

JetPack 4.5 (based on L4T 32.5) introduces a number of components and features that were not present in JetPack 4.2 (L4T 32.1), several of which are directly relevant to the monocular camera pipeline and onboard perception stack used on the SparkFun JetBot's Jetson Nano.

- **VPI (Vision Programming Interface) 1.0** – first production release of NVIDIA's hardware-accelerated computer vision library, offering CPU, GPU, and PVA backends as an alternative or complement to OpenCV for feature extraction.
- **Extended Multimedia API (V4L2) for CSI cameras** – improved direct control over encoding, decoding, latency, bit-rate, and image quality for CSI-connected cameras such as the JetBot's front-mounted camera.
- **Enhanced secure boot and disk encryption** – encryption extended to kernel, kernel-dtb, and initrd images, with added at-rest disk encryption.
- **Redundant filesystem management** – support for maintaining and updating two redundant root filesystems for safer over-the-air style updates.
- **New NFS-based flashing method** – enables flashing eMMC, SD, USB, and NVMe storage devices over a network filesystem.
- **Updated U-Boot (v2020.04)** – along with general boot-performance improvements.
- **Jetson Nano bootloader relocation** – boot firmware moved to onboard QSPI-NOR flash, freeing the microSD card for OS and application storage only.
- **DeepStream 5.0 support** added to the SDK component stack.

The underlying software stack was also substantially upgraded between the two releases:

Table 9.1: Core library versions: JetPack 4.2 vs. JetPack 4.5

Component	JetPack 4.2	JetPack 4.5
L4T	32.1.0	32.5
CUDA	10.0.166	10.2.89
cuDNN	7.3.1	8.0.0
TensorRT	5.0.6.3	7.1.3.0
OpenCV	3.3.1 (no CUDA)	4.1.1
VPI	Not available	1.0
Docker	Not standard	19.03
DeepStream	Not supported	5.0

JetPack 4.5 introduces several components absent in JetPack 4.2, including the first production release of NVIDIA's Vision Programming Interface (VPI 1.0), an extended V4L2 Multimedia API for CSI camera control, enhanced secure boot with disk encryption, redundant filesystem management, a new NFS-based flashing method, an updated U-Boot (v2020.04), and relocation of the Jetson Nano bootloader to onboard QSPI-NOR flash. The underlying stack was also upgraded, moving from CUDA 10.0/cuDNN 7.3/TensorRT 5.0/ OpenCV 3.3 in JetPack 4.2 to CUDA 10.2/cuDNN 8.0/TensorRT 7.1.3/OpenCV 4.1.1 in JetPack 4.5 [NVI21].

Bibliography

[CCC+16] C. Cadena *et al.*, "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1309–1332, 2016. DOI: 10.1109/TRO.2016.2624754.

[MBR17] L. Matthies, M. Bajracharya, and A. Rankin, "Visual perception for autonomous indoor navigation and obstacle avoidance," *Journal of Field Robotics*, vol. 34, no. 1, pp. 1–20, 2017, ISSN: 1556-4959.

[NVI19] NVIDIA Corporation, *Nvidia jetson nano developer kit: Data sheet*, Technical datasheet, 2019. [Online]. Available: https://cdn.sparkfun.com/assets/0/7/f/9/d/jetson-nano-devkit-datasheet-updates-us-v3.pdf.

[NVI20] NVIDIA Corporation, *Jetson nano developer kit user guide*, Online manual, 2020. [Online]. Available: https://developer.nvidia.com/embedded/downloads.

[NVI21] NVIDIA Corporation, *Jetson linux (l4t) and jetpack software overview*, Online documentation, 2021. [Online]. Available: https://developer.nvidia.com/embedded/jetpack.

[SNS11] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA, USA: MIT Press, 2011, ISBN: 978-0-262-01535-6.

[Spa19a] SparkFun Electronics, *Sparkfun qwiic motor driver*, Online product page, 2019. [Online]. Available: https://www.sparkfun.com/products/15451.

[Spa19b] SparkFun Electronics, *Sparkfun qwiic phat v2.0 for raspberry pi*, Online product page, 2019. [Online]. Available: https://www.sparkfun.com/sparkfun-qwiic-phat-v2-0-for-raspberry-pi.html.

[Spa20a] SparkFun Electronics, *Assembly guide for sparkfun jetbot ai kit v2.0*, Online tutorial, 2020. [Online]. Available: https://learn.sparkfun.com/tutorials/assembly-guide-for-sparkfun-jetbot-ai-kit-v20/all.

[Spa20b] SparkFun Electronics, *Sparkfun jetbot AI kit v2.0 powered by jetson nano*, Online product page, 2020. [Online]. Available: https://www.sparkfun.com/sparkfun-jetbot-ai-kit-v2-0-powered-by-jetson-nano.html.

[Zuf+24] R. N. Zufar *et al.*, "Selection of lightweight cnn models with limited resources for collision avoidance systems," *ECTI-EEC*, 2024. [Online]. Available: `https://ph02.tci-thaijo.org/index.php/ECTI-EEC/article/download/251164/170275/927583`.